Team Details

- ➔ Team name – Tactical Titans
- ➔ Team leader name – Sathvik Bhushan G
- ➔ Problem Statement – AI for Smart Cities

Brief about the idea

- The project is an vehicle recognition and automated access control system designed for offices, corporate campuses, and gated residential communities. The idea is to replace manual gate entry processes with a fast, secure, and AI-driven solution that can identify vehicles and make instant access decisions.
- In traditional setups, security guards manually check vehicles, write entries in registers, and call residents to confirm visitors. This process is slow, prone to errors, and difficult to monitor. The project automates this entire workflow using a camera installed at the entry point. The system captures the vehicle's number plate and uses AI-based number plate recognition to extract the registration number in real time.
- The extracted number is then verified against a pre-approved database containing residents, employees, staff, and registered visitors. If the vehicle is authorized, the gate opens automatically and the entry is logged digitally. If the vehicle is not found in the database, access is denied, the attempt is recorded, and security personnel are alerted. Repeat unauthorized attempts are flagged as potential intruder activity.
- The system maintains a complete digital log of all vehicle movements, enabling monitoring, auditing, and traffic analysis. It reduces dependency on manual guards, minimizes entry delays during peak hours, and improves overall security.
- The idea is to transform traditional gate security into a smart, automated, and data-driven access management system.

Opportunities

- ➔ How different is it from any of the other existing ideas?
- ➔ How will it be able to solve the problem?
- ➔ USP of the proposed solution

The project is different because it is designed as a complete, decision-making system — not just a monitoring tool. It does not simply read number plates; it automatically verifies, decides, logs and alerts in real time.
The project uses automating verification , eliminates manual checking errors , makes real-time decisions , every vehicle entry is recorded and searchable and it detects repeat unauthorized attempts .

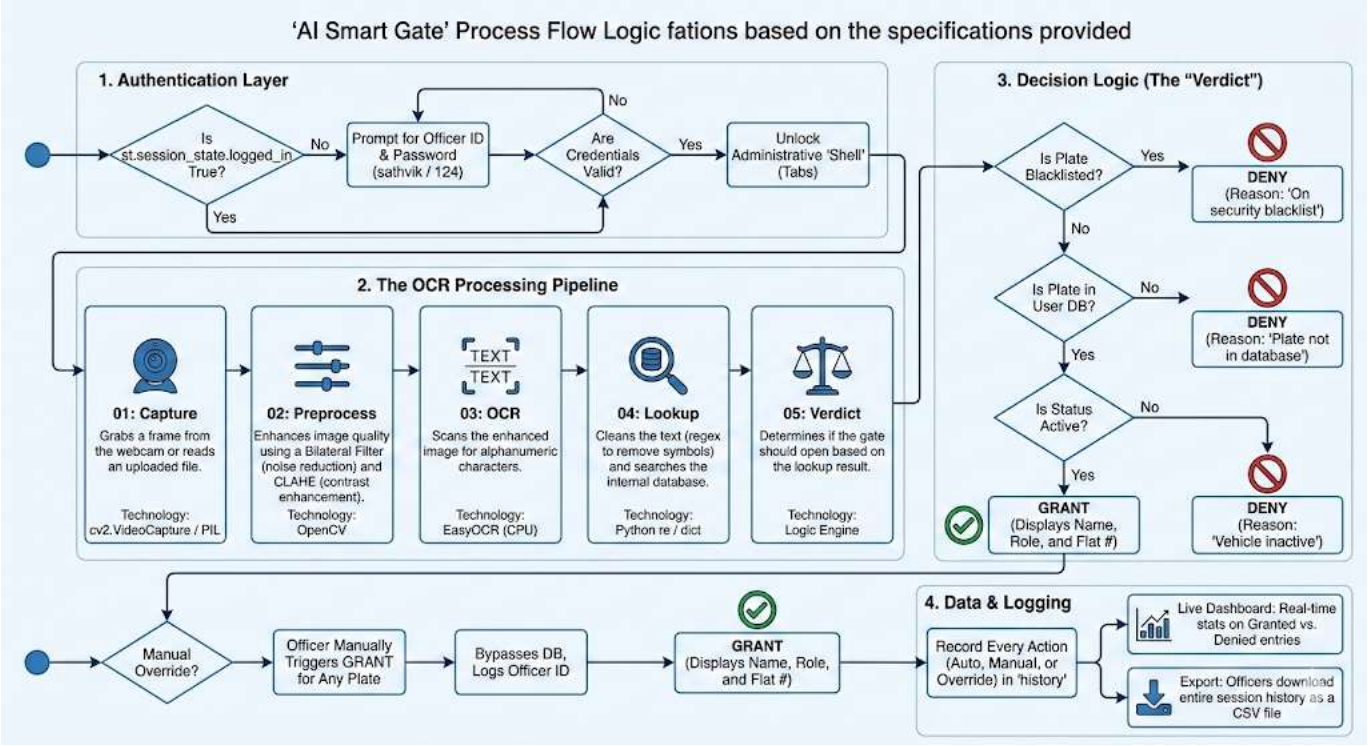
USP:

Edge-Based Real-Time Processing – Instant decisions with minimal latency.
Integrated Intruder Intelligence – Not just detection, but behavioral tracking.
Fully Automated Gate Control – No manual verification required.
Scalable Architecture – Works for one gate or hundreds of gates.
Privacy-Focused Design – No unnecessary cloud dependency.
Data-Driven Security Insights – Provides analytics, reports, and audit trails.

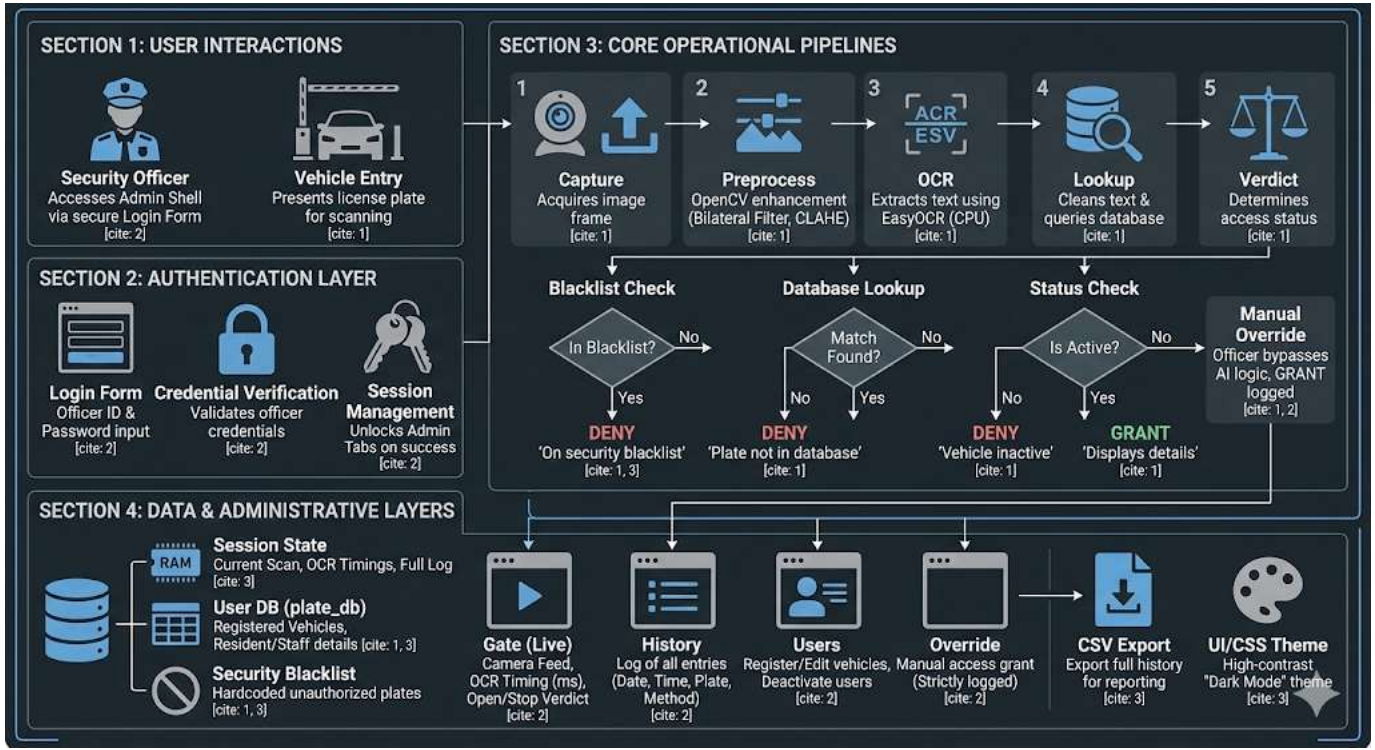
List of features offered by the solution

Real-time vehicle detection (cars and bikes)
AI-based automatic number plate recognition (ANPR)
Instant database verification of vehicle registration numbers
Automated gate open/close control for authorized vehicles
Immediate access denial for unauthorized vehicles
OCR confidence scoring for plate detection accuracy
Blacklist/watchlist vehicle support
Live access log with time-stamped entries
Vehicle type categorization (Resident / Visitor / Staff / Unknown)
Daily entry statistics (Total / Authorized / Denied)
Repeat intruder detection and flagging
Real-time security alerts for denied entries
Blacklist/watchlist vehicle support
Tamper or unreadable plate detection
Tailgating detection (prevent unauthorized vehicles from following in)

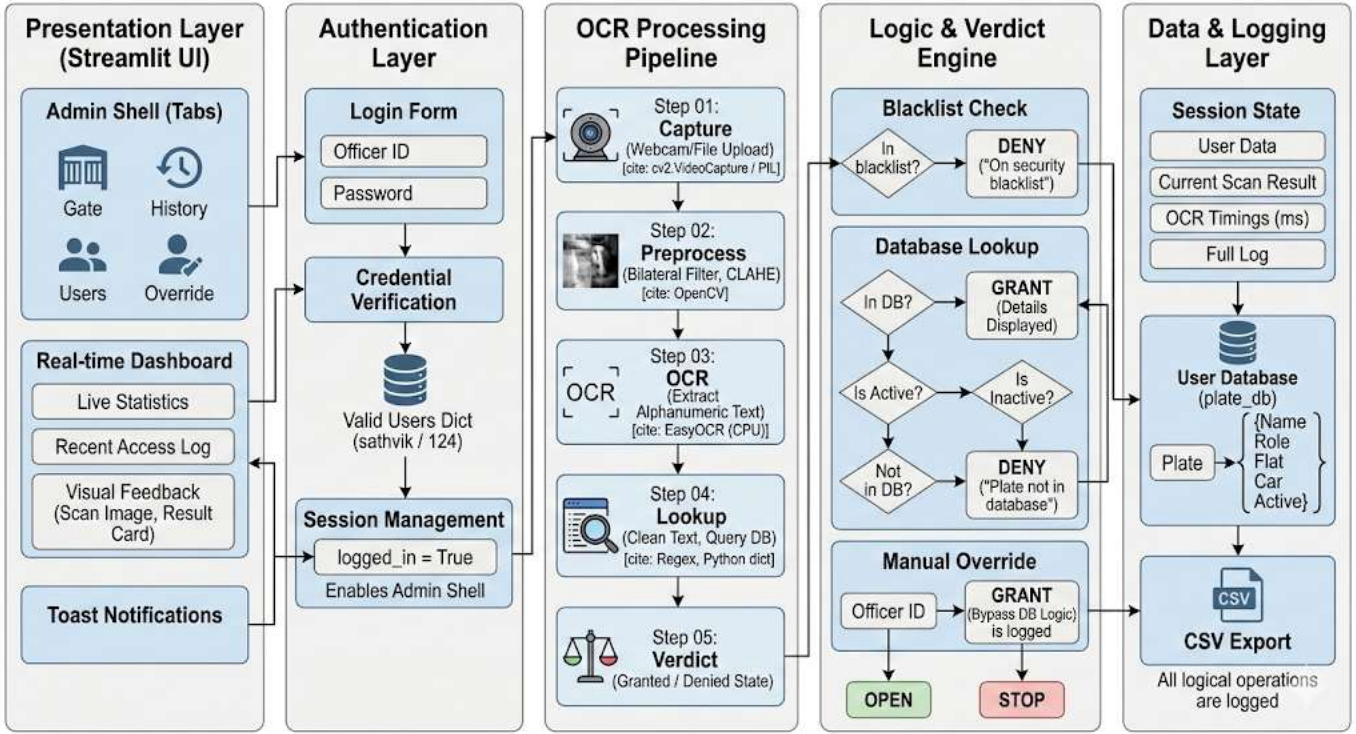
Process flow diagram or Use-case diagram



Wireframes/Mock diagrams of the proposed solution (optional)



Architecture diagram of the proposed solution



Technologies to be used in the solution

1. Computer Vision & Image Processing
•**OpenCV (cv2)**: The primary library for handling visual data. It is used to capture frames from the webcam and perform critical preprocessing steps like:
- •**Bilateral Filtering**: To reduce image noise while keeping edges sharp.
- •**CLAHE (Contrast Limited Adaptive Histogram Equalization)**: To improve visibility in low-light or high-glare conditions.

•**Pillow (PIL)**: Used for basic image manipulation and handling file uploads within the Streamlit interface.

2. Machine Learning & OCR
•**EasyOCR**: This is the "brain" of the solution. It utilizes Deep Learning (specifically a CRAFT text detector and a CRNN recognition model) to identify and extract alphanumeric characters from license plates. It is configured in the code to run on the **CPU** for maximum compatibility.
•**Regular Expressions (re)**: A vital "cleaning" layer that uses pattern matching to remove non-alphanumeric noise from the OCR output, ensuring the data matches the database format.

3. Frontend & Application Framework
•**Streamlit**: A Python-based framework used to build the entire dashboard. It handles:
- •**Reactive UI**: Updating the "Verdict" card immediately after a scan.
- •**Session State**: Storing temporary data like the current scan result and login status without needing a heavy database.
- •**Layout**: Creating the dark-themed, administrative multi-tab interface (Gate, History, Users, Override).

4. Data Management & Connectivity
•**Python Dictionaries**: Used as an in-memory database (plate_db) for fast lookups.
•**CSV Module**: Handles data persistence by allowing officers to export the session history for external reporting.
•**Certifi & SSL**: Used during the initial setup to securely download the pre-trained ML models for EasyOCR.

5. Deployment Environment
•**Python 3.x**: The core programming language.
•**Hardware**: A standard PC or laptop with a webcam for real-time video capture, though the code is also compatible with static image uploads.

Usage of AMD Products/Solutions

The Following AMD Products/Solutions can be replaced with the current tech to scale and launch the product

Layer	Standard Tech	AMD Solution
Pre-processing	OpenCV (CPU)	**OpenCV with OpenCL** (Accelerated via Radeon)
OCR Engine	EasyOCR (CPU)	**PyTorch (ROCm)** or **Ryzen AI (NPU)**
Logic/Verdict	Python Logic	**AMD EPYC** (for high-concurrency cloud scaling)
Data Storage	CSV/Local	**AMD Instinct™** (for large-scale historical analytics)

Estimated implementation cost (optional)

The cost that would be incurred by using standard tech would be only the cost required to set up a camera . This would work if it were to be used for a single gate .
The implementation cost changes when we try to incorporate AMD Products/Solutions

1. Single-Gate Edge Kiosk (AMD Ryzen™ AI)

Target: Apartment Complexes, Small Office Entrances.

Category	Component	Specification	Estimated Cost (INR)
Compute	**AMD Mini-PC**	Ryzen 7 8845HS (NPU Enabled)	₹48,000 – ₹55,000
Vision	**LPR Camera**	4MP PoE Bullet (Hikvision/Dahua LPR)	₹12,000 – ₹15,000
Hardware	**I/O Relay**	USB/Ethernet Relay for Gate Motor	₹2,500 – ₹3,500
Enclosure	**Outdoor Kiosk**	MS Powder Coated, Weatherproof	₹8,000 – ₹12,000
Networking	**PoE & Wiring**	Cat6 Cable + PoE Injector	₹3,000 – ₹4,500
TOTAL			**~₹73,500 – ₹90,000**

2. Multi-Lane Enterprise Solution (AMD Radeon™ + ROCm)

Target: Large IT Parks (Bengaluru/Hyderabad), Airports, Metro Parking. A centralized server using **AMD ROCm** to process up to 8–10 lanes simultaneously from a single workstation.

Category	Component	Specification	Estimated Cost (INR)
Server	**AMD Workstation**	Ryzen 9 7950X, 64GB DDR5 RAM	₹1,10,000 – ₹1,25,000
Acceleration	**AMD Radeon™ Pro**	Radeon Pro W7800 (32GB VRAM)	₹1,65,000 – ₹1,80,000
Cameras	**4x IP Cameras**	4K High-Speed Global Shutter	₹60,000 – ₹80,000
Installation	**Mounts & Cabling**	Industrial Grade Setup	₹15,000 – ₹25,000
TOTAL			**~₹3,50,000 – ₹4,10,000**

Prototype Assets (Optional)

- ➔ GitHub Public Repository Link
- ➔ Demo Video Link (Max: 3 Minutes)

https://github.com/sathvikg08/amdslingshot-26

https://drive.google.com/file/d/1C5GEOlK5dO-qJPElF-2ferREDa8uBS0R/view?usp=drive_link

Add as per the requirements of the contest

Type here

AMD Slingshot

HUMAN *IMAGINATION*
BUILT WITH *AI*

Powered by H2S

Thank you!